4 Interpolation and approximation

4.1 Interpolation — 4.2 Lagrange interpolation — 4.3 Newton interpolation — 4.4 Hermite interpolation — 4.5 Piecewise cubic Hermite interpolation — 4.6 Approximation — 4.7 Least squares fitting — 4.8 Improving the parameter — 4.9 Problems

In geometric modeling and other applications, one often needs to find an analytic, usually polynomial, representation of curves for which no mathematical or only complicated descriptions are known. To construct such a representation, one usually measures or evaluates a given curve at a number of points and determines an interpolant or approximant. This chapter reviews some basic techniques.

4.1 Interpolation

A sequence of n functions $C_1(u), \ldots, C_n(u)$ is said to be linearly independent over the sequence $u_1, \ldots, u_n$ if the matrix

$$C = \begin{bmatrix} C_1(u_1) & \cdots & C_n(u_1) \\ \vdots & & \vdots \\ C_1(u_n) & \cdots & C_n(u_n) \end{bmatrix}$$

is non-singular. In this case, to any n points $\mathbf{p}_1, \ldots, \mathbf{p}_n \in \mathbb{R}^d$ there is a unique curve

$$\mathbf{p}(u) = \sum_{i=1}^{n} \mathbf{x}_i C_i(u)$$

interpolating the points $\mathbf{p}_i$ at the nodes u_i, i.e.,

$$\mathbf{p}(u_i) = \mathbf{p}_i \ , \quad i = 1, \ldots, n \ .$$

H. Prautzsch et al., *Bézier and B-Spline Techniques*
© Springer-Verlag Berlin Heidelberg 2002

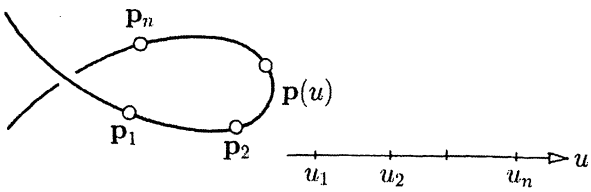

Figure 4.1: Interpolating curve

For a proof, write the **interpolation conditions** in matrix notation,

$$\begin{bmatrix} C_1(u_1) & \dots & C_n(u_1) \\ \vdots & & \vdots \\ C_1(u_n) & \dots & C_n(u_n) \end{bmatrix} \begin{bmatrix} \mathbf{x}_1^t \\ \vdots \\ \mathbf{x}_n^t \end{bmatrix} = \begin{bmatrix} \mathbf{p}_1^t \\ \vdots \\ \mathbf{p}_n^t \end{bmatrix} ,$$

or more concisely,

$$CX = P ,$$

which represents d simultaneous linear systems for the d columns of X. The existence of the solution follows from the linear independence of $C_1, \dots, C_n$ over $u_1, \dots, u_n$.

Remark 1: If the C_i are linearly independent polynomials of degree $n-1$, then the matrix C is invertible for any n distinct values $u_1, \dots, u_n$. Namely, the homogeneous system $C\mathbf{x} = \mathbf{o}$ (for a single column $\mathbf{x}$) has only the trivial solution $\mathbf{x} = \mathbf{o}$ since the zero polynomial is the only polynomial of degree $n-1$ with n roots.

Remark 2: Two points can be interpolated by a line, three points by a parabola, four points by a cubic, and so on.

4.2 Lagrange form

A fundamental and simple method to obtain a polynomial interpolant is due to Lagrange. Given $n+1$ points $\mathbf{p}_i$ with corresponding parameter values u_i, $i = 0, \dots, n$, the interpolating polynomial curve of degree n is

$$\mathbf{p}(u) = \sum_{i=0}^{n} \mathbf{p}_i L_i^n(u) ,$$

where the **Lagrange polynomials** $L_i^n(u), i = 0, \ldots, n$, are defined by the interpolation conditions

$$L_i^n(u_k) = \begin{cases} 1 & \text{if} \quad k = i \\ 0 & \phantom{\text{if}} \quad k \neq i \end{cases} .$$

Figure 4.2 shows an example.

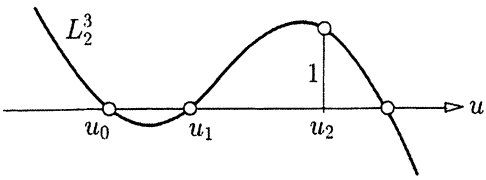

Figure 4.2: A cubic Lagrange polynomial.

Checking the definition, we easily verify that

$$L_i^n(u) = \frac{\prod_{j=0,\, j\neq i}^{n}(u - u_j)}{\prod_{j=0,\, j\neq i}^{n}(u_i - u_j)} .$$

There are different ways to evaluate Lagrange polynomials. One possibility is to use the following recursion. First observe that the Lagrange polynomials sum to one,

$$\sum_{i=0}^{n} L_i^n(u) \equiv 1 .$$

Then it follows from the definition of the Lagrange polynomials that

$$(1) \quad L_i^k = \begin{cases} L_i^{k-1}\alpha_{ik} & \text{for} \quad i = 0, \ldots, k-1 \\ 1 - \sum_{j=0}^{k-1} L_j^k = \sum_{j=0}^{k-1} L_j^{k-1}(1 - \alpha_{jk}) & \text{for} \quad i = k \ , \end{cases}$$

where α_{ik} represents the local parameter over $[u_k, u_i]$, i.e.,

$$\alpha_{ik} = \frac{u - u_k}{u_i - u_k} .$$

In analogy to the derivation of de Casteljau's algorithm in 2.3, one obtains from recurrence relation (1) an iterative method to compute $\mathbf{p}(u)$ from the points

$$\mathbf{p}_i^0 = \mathbf{p}_i$$

by affine combinations:

$$\begin{aligned}
\mathbf{p}(u) &= \sum_{i=0}^{n} \mathbf{p}_i^0 L_i^n(u) \\
&= \sum_{i=0}^{n-1} \mathbf{p}_i^1 L_i^{n-1}(u) \\
&\vdots \\
&= \sum_{i=0}^{0} \mathbf{p}_i^n L_i^0(u) = \mathbf{p}_0^n \ ,
\end{aligned}$$

where

$$\mathbf{p}_i^{k+1} = \mathbf{p}_i^k \alpha_{i,n-k} + \mathbf{p}_{n-k}^k (1 - \alpha_{i,n-k}) \ .$$

This computation of $\mathbf{p}(u)$ is known as **Aitken's algorithm**. The polynomials $\mathbf{p}_i^k = \mathbf{p}_i^k(u)$ have degree $\leq k$ and interpolate the points $\mathbf{p}_i$ and $\mathbf{p}_n, \cdots, \mathbf{p}_{n-k+1}$.

Remark 3: The computation of the Lagrange polynomials by recursion formula(1) can be conveniently arranged in a triangular array,

$$\begin{array}{lccccc}
1 = & L_0^0 & L_0^1 & L_0^2 & \cdots & L_0^n \\
 & & L_1^1 & L_1^2 & \cdots & L_1^n \\
 & & & L_2^2 & \cdots & L_2^n \\
 & & & & \ddots & \vdots \\
 & & & & & L_n^n \ .
\end{array}$$

Remark 4: Similarly, one can arrange the computation of the $\mathbf{p}_i^k$ in a triangular array,

$$\begin{array}{lllll}
\mathbf{p}_0^0 & & & & \\
\mathbf{p}_1^0 & \mathbf{p}_0^1 & & & \\
\mathbf{p}_2^0 & \mathbf{p}_1^1 & \mathbf{p}_0^2 & & \\
\vdots & & & \ddots & \\
\mathbf{p}_n^0 & \mathbf{p}_{n-1}^1 & \mathbf{p}_{n-2}^2 & \cdots & \mathbf{p}_0^n \ .
\end{array}$$

4.3 Newton form

Another polynomial basis, also depending on the interpolation abscissae, has been introduced by Newton, namely the $n+1$ monic polynomials $P_i(u)$ of degree i with zeroes at the first i interpolation abscissae $u_0, \ldots, u_{i-1}$,

$$P_0 = 1 \quad \text{and, for} \quad i \geq 1, \quad P_i(u) = (u - u_0) \cdots (u - u_{i-1}) \ .$$

Let $[u_0 \ldots u_i]\mathbf{q}$ denote the leading coefficient of the ith degree polynomial that interpolates some given curve $\mathbf{q} = \mathbf{q}(u)$ at $u_0, \ldots, u_i$. Then, it follows that the polynomial curve $\mathbf{p}(u)$ of degree $\leq n$ interpolating $\mathbf{q}(u)$ at $u_0, \ldots, u_n$ can be written as

$$\mathbf{p}(u) = \sum_{i=0}^{n} [u_0 \ldots u_i]\mathbf{q} \cdot P_i(u) \ .$$

From the Lagrange representation, it follows that the leading coefficient is of the form

$$[u_0 \ldots u_n]\mathbf{q} = \sum_{k=0}^{n} \frac{\mathbf{q}(u_k)}{(u_k - u_0) \cdots (u_k - u_k)^* \cdots (u_k - u_n)} \ ,$$

where the asterisk * means the absence of this term. Using this explicit representation, it is easy to verify the recursion formula

$$\begin{aligned} [u_0 \ldots u_n]\mathbf{q} &= \frac{[u_1 \ldots u_n]\mathbf{q} - [u_0 \ldots u_{n-1}]\mathbf{q}}{u_n - u_0} \\ &\vdots \\ [u_i\, u_j]\mathbf{q} &= \frac{[u_j]\mathbf{q} - [u_i]\mathbf{q}}{u_j - u_i} \\ [u_j]\mathbf{q} &= \mathbf{q}(u_j) \ . \end{aligned}$$

Because of this recursion, $[u_0 \ldots u_i]\mathbf{q}$ is called the ith **divided difference** of $\mathbf{q}(u)$ at $u_0, \ldots, u_i$. Note that the divided difference $[u_0 \ldots u_n]\mathbf{q}$ is symmetric with respect to its knots $u_0, \ldots, u_n$.

Remark 5: If $\mathbf{q}(u)$ is sufficiently smooth, then the divided difference $[u_0 \ldots u_n]\mathbf{q}$ depends continuously on the abscissae u_i, see Problems 1 and 2. Consequently, $[u_0 \ldots u_n]\mathbf{q}$ is also defined if some or all u_i coalesce. In particular, one has

$$[u_i \overset{n+1}{\ldots} u_i]\mathbf{q} = \mathbf{q}^{(n)}(u_i)/n! \ .$$

Remark 6: For $u_0 = \cdots = u_k$ it follows from Remark 5 that

$$\mathbf{p}(u) = \sum_{j=0}^{n} [u_0 \ldots u_j]\mathbf{q} \cdot P_j(u)$$

interpolates all derivatives up to order k of $\mathbf{q}(u)$ at $u = u_0$. Since $\mathbf{p}(u)$ does not depend on the order of the abscissae u_i, it also interpolates any derivative $\mathbf{q}^{(r)}(u_i)$, where $u_i = \cdots = u_{i+r}$.

Remark 7: If the values u_i are equidistant, $u_i = u_0 + ih$, then it follows that

$$[u_0 \ldots u_n]\mathbf{q} = \frac{1}{n!\,h^n}\Delta^n \mathbf{q}(u_0) \ .$$

4.4 Hermite interpolation

From Remark 6 in Section 4.3, it follows that the curve

$$\mathbf{p}(u) = \sum_{i=0}^{n} [u_0 \ldots u_i]\mathbf{q} \cdot P_i$$

interpolates points and, if interpolation abscissae coalesce, also derivatives of $\mathbf{q}$. This type of interpolation is called **Hermite interpolation**.

In particular, it is very common to interpolate all derivatives up to some order k at the end points of some interval $[a, b]$. The corresponding interpolant is of degree $n = 2k + 1$ and can be written as

$$\mathbf{p}(u) = \sum_{i=0}^{k} \mathbf{q}_a^{(i)} H_i^n + \sum_{i=0}^{k} \mathbf{q}_b^{(i)} H_{n-i}^n \ ,$$

where $\mathbf{q}_u^{(i)}$ denotes the ith derivative of $\mathbf{q}$ at u and where the so-called **Hermite polynomials** H_i^n of degree n are defined by

$$\frac{d^r}{du^r} H_i^n(a) = \frac{d^r}{du^r} H_{n-i}^n(b) = \begin{cases} 1 & \text{if} \quad i = r \\ 0 & \text{otherwise} \end{cases} \quad , \quad r = 0, \ldots, k \ .$$

Figure 4.3 shows a cubic Hermite interpolant and the corresponding four cubic Hermite polynomials.

More conveniently, one can describe the interpolant by its Bézier points $\mathbf{b}_j$ over $[a, b]$. From Section 2.8, it follows that

$$\mathbf{b}_j = \sum_{i=0}^{j} \binom{j}{i} \frac{(b-a)^i}{n \cdots (n-i+1)} \mathbf{q}_a^{(i)}$$

and

$$\mathbf{b}_{n-j} = \sum_{i=0}^{j} \binom{j}{i} \frac{(a-b)^i}{n \cdots (n-i+1)} \mathbf{q}_b^{(i)} \ ,$$

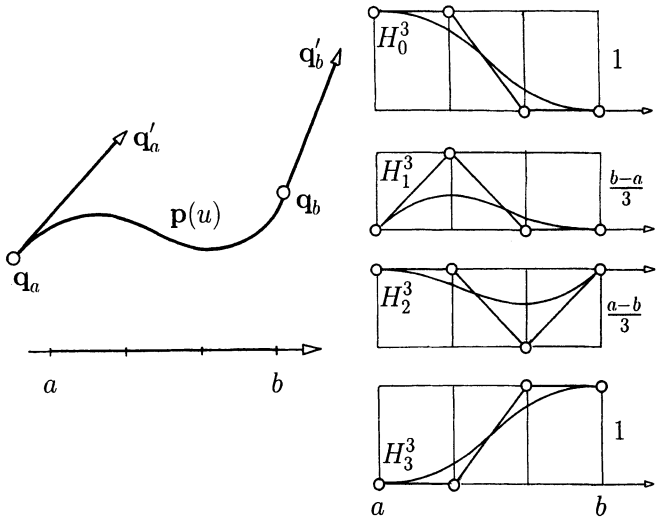

Figure 4.3: Cubic Hermite interpolation.

i.e., for $j = 0, \ldots, (n-1)/2$, one has

$$\begin{aligned}
\mathbf{b}_0 &= \mathbf{q}_a \\
\mathbf{b}_1 &= \mathbf{q}_a + \frac{b-a}{n}\,\mathbf{q}'_a \\
\mathbf{b}_2 &= \mathbf{q}_a + \frac{b-a}{n}\,(2\mathbf{q}'_a + \frac{b-a}{n-1}\,\mathbf{q}''_a) \\
&\vdots \\
\mathbf{b}_n &= \mathbf{q}_b\ .
\end{aligned}$$

Remark 8: Instead of interpolating the second derivatives $\mathbf{q}''_a$ and $\mathbf{q}''_b$, one can interpolate the curvatures at a and b. The **curvature** at a is given by

$$x_a = \frac{n-1}{n}\,\frac{\|\,\Delta\mathbf{b}_0 \times \Delta^2\mathbf{b}_0\,\|}{\|\,\Delta\mathbf{b}_0\,\|^3}\ ,$$

where $\times$ denotes the cross product.

4.5 Piecewise cubic Hermite interpolation

A function $f(u)$ can be approximated by a polynomial p of degree n interpolating f at some given abscissae $u_0 < \cdots < u_n$. A standard result from numerical analysis states that the deviation of p from f over $[u_0, u_n]$ can be

written as

$$p(u) - f(u) = \frac{f^{(n+1)}(v)}{(n+1)!}(u-u_0)\cdots(u-u_n) \ ,$$

where $v = v(u)$ lies in $[u_0, u_n]$. Thus, the error becomes smaller if the abscissae move closer. On the other hand, a higher degree polynomial interpolating f at more points is, in general, not a better approximant. Therefore, one resorts to piecewise polynomials of low degree.

For example, cubics are very popular since they are of low degree and, for many applications, of sufficient flexibility. In the sequel, we describe the piecewise cubic Hermite interpolation.

Given $m+1$ points $\mathbf{p}_0, \ldots, \mathbf{p}_m$ and derivatives $\mathbf{d}_0, \ldots, \mathbf{d}_m$ with corresponding parameter values $u_0 < \cdots < u_m$, there is a unique piecewise cubic C^1 curve $\mathbf{s}(u)$ over $[u_1, u_m]$ such that

$$\mathbf{s}(u_i) = \mathbf{p}_i \ , \quad \text{and} \quad \mathbf{s}'(u_i) = \mathbf{d}_i \ .$$

Its Bézier representation is readily obtained from Section 4.4, and it is

$$\mathbf{s}(u) = \sum_{j=0}^{3} \mathbf{b}_{3i+j} B_j^3(t_i) \ , \quad u \in [u_i, u_{i+1}] \ ,$$

where $t_i = (u-u_i)/(u_{i+1}-u_i)$ represents the local parameter over $[u_i, u_{i+1}]$, $i = 0, \ldots, m-1$, and

$$\begin{aligned} \mathbf{b}_{3i} &= \mathbf{p}_i \\ \mathbf{b}_{3i+1} &= \mathbf{p}_i + \mathbf{d}_i \Delta u_i/3 \\ \mathbf{b}_{3i+2} &= \mathbf{p}_{i+1} - \mathbf{d}_{i+1} \Delta u_i/3 \ , \end{aligned}$$

see Figure 4.4 for an illustration.

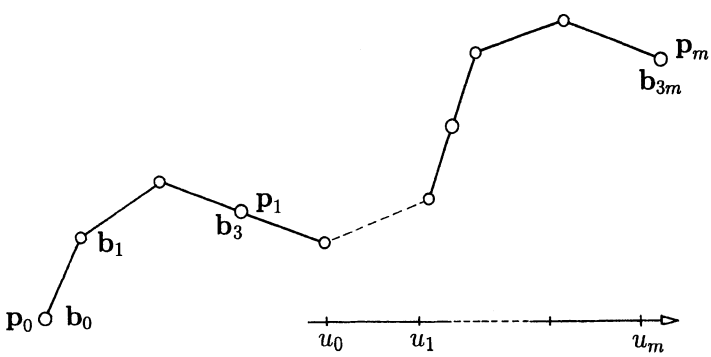

Figure 4.4: Composite Bézier polygon.

Often, the derivatives $\mathbf{d}_i$ are not given, but have to be determined from the data points. A simple method to estimate the $\mathbf{d}_i$ is to compute the derivatives of the quadratic interpolant to the three consecutive points $\mathbf{p}_{i-1}, \mathbf{p}_i, \mathbf{p}_{i+1}$, as illustrated in Figure 4.5.

This means to set

$$\mathbf{d}_i = (1-\alpha_i)\,\frac{\Delta\mathbf{p}_{i-1}}{\Delta u_{i-1}} + \alpha_i \frac{\Delta\mathbf{p}_i}{\Delta u_i}, \quad \text{for} \quad i = 1, \ldots, m-1 \ ,$$

where

$$\alpha_i = \frac{\Delta u_{i-1}}{\Delta u_{i-1} + \Delta u_i} \ .$$

At the endpoints, one adjusts these estimations to

$$\mathbf{d}_0 = 2\,\frac{\Delta\mathbf{p}_0}{\Delta u_0} - \mathbf{d}_1 \ , \quad \text{and} \quad \mathbf{d}_m = 2\,\frac{\Delta\mathbf{p}_{m-1}}{\Delta u_{m-1}} - \mathbf{d}_{m-1} \ .$$

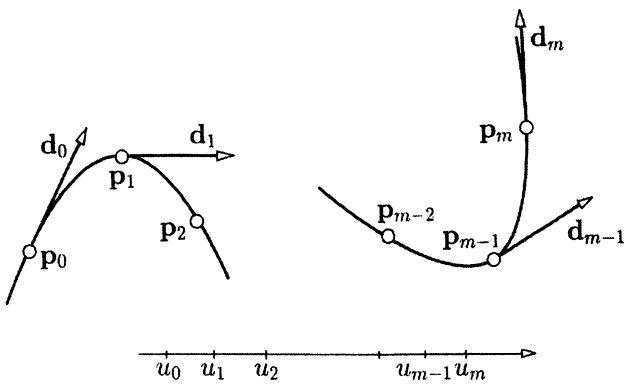

Figure 4.5: Obtaining derivative estimates from parabolas.

One often also must determine the interpolation abscissae u_i. Some simple choices are

the **equidistant parametrization**, for which

$$\Delta u_i = \text{constant} \ ,$$

the **chord length parametrization**, for which

$$\Delta u_i = \|\Delta\mathbf{p}_i\| \ ,$$

and the **centripetal parametrization** [Lee '89], for which

$$\Delta u_i = \sqrt{\|\Delta \mathbf{p}_i\|} \ .$$

These and further methods are discussed at length in [Foley & Nielson '89, Farin '02, Hoschek & Lasser '92].

Remark 9: An interpolation scheme with fixed interpolation abscissae u_i is called **linear** if the Bézier points $\mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_{3m}$ depend linearly on the interpolation points $\mathbf{p}_0, \ldots, \mathbf{p}_m$. Thus, a **linear interpolation scheme** is given by an $m{+}1 \times 3m{+}1$ matrix S that establishes the relationship between the $\mathbf{b}_i$ and $\mathbf{p}_k$

$$[\mathbf{b}_0 \ldots \mathbf{b}_{3m}] = [\mathbf{p}_0 \cdots \mathbf{p}_m]S \ .$$

Note that the interpolation scheme above, with the derivatives $\mathbf{d}_i$ obtained from quadratic interpolants, is linear.

4.6 Approximation

In general, given any sequence of n basis functions $C_1, \ldots, C_n$, there is no curve

$$\mathbf{p}(u) = \sum_{i=1}^{n} \mathbf{x}_i C_i(u)$$

that interpolates more than n points $\mathbf{p}_1, \ldots, \mathbf{p}_m$, $m > n$, at prescribed parameter values $u_1, \ldots, u_m$. In this case, one determines an **approximant** $\mathbf{p}(u)$, usually under the condition that the differences

$$\mathbf{p}(u_i) - \mathbf{p}_i = \mathbf{r}_i$$

become minimal in some sense.

Using matrix notation, this means to minimize

$$\begin{bmatrix} C_1(u_1) & \ldots & C_n(u_1) \\ \vdots & & \vdots \\ C_1(u_m) & \ldots & C_n(u_m) \end{bmatrix} \begin{bmatrix} \mathbf{x}_1^t \\ \vdots \\ \mathbf{x}_n^t \end{bmatrix} - \begin{bmatrix} \mathbf{p}_1^t \\ \vdots \\ \mathbf{p}_m^t \end{bmatrix} = \begin{bmatrix} \mathbf{r}_1^t \\ \vdots \\ \mathbf{r}_m^t \end{bmatrix}$$

or, more concisely,

$$CX - P = R = [r_{ij}] \ .$$

There are several choices that can be used to measure the "size" of R. Most

commonly one minimizes the square sum

$$\sum_{i=1}^{m} \mathbf{r}_i^2 = \sum_{i,j} r_{ij}^2 = \operatorname{trace} R^t R = \operatorname{trace} R R^t \ .$$

This method is due to Gauss and it is called **least squares fitting**.

A slight modification of this approach is to minimize the weighted square sum

$$\sum w_i^2 \mathbf{r}_i^2 = \operatorname{trace} (WR)^t (WR) \ ,$$

with some weight matrix

$$W = \begin{bmatrix} w_1 & & \\ & \ddots & \\ & & w_m \end{bmatrix} \ .$$

Both approaches result in linear systems as discussed in the next section. Other approaches are more complicated. For example, one could minimize the maximum distance

$$\max \|\mathbf{r}_i\|_\infty = \max \|r_{ij}\| \ .$$

This leads to a system of linear inequalities, which can be solved by the simplex algorithm, see Problem 8.

Remark 10: It is important to note that the jth column of R only depends on the jth column of X for all j. Therefore, minimizing R using any of the three distances above means to minimize each column of R separately.

4.7 Least squares fitting

In this section, we will show how to find a least squares fit. However, more generally than in the previous section, we also require the solution X to satisfy further linear constraints $DX = Q$, which would typically force the approximant $\mathbf{p}(u)$ to interpolate some of the points $\mathbf{p}_i$. As explained in Remark 10 of 4.6, it suffices to consider a column $\mathbf{x}$ of X and the corresponding columns $\mathbf{p}, \mathbf{q}$, and $\mathbf{r}$ of P, Q, and R, respectively.

The following theorem shows how one can find the minimal **residual**

$$\mathbf{r} = C\mathbf{x} - \mathbf{p}$$

under the constraints $D\mathbf{x} = \mathbf{q}$.

The sum of squares $\mathbf{r}^t\mathbf{r} = \sum r_i^2$ *is minimal for the solution* $\mathbf{x}, \mathbf{y}$

of the linear system

$$\left[\begin{array}{c|c} C^tC & D^t \\ \hline D & O \end{array}\right] \left[\begin{array}{c} \mathbf{x} \\ \hline \mathbf{y} \end{array}\right] = \left[\begin{array}{c} C^t\mathbf{p} \\ \hline \mathbf{q} \end{array}\right] . \tag{2}$$

Proof: Let $\mathbf{x}, \mathbf{y}$ be a solution of (2) such that $D\mathbf{x} = \mathbf{q}$. Further, let $\mathbf{x} + \mathbf{h}$ be any other point satisfying the constraints

$$D[\mathbf{x} + \mathbf{h}] = \mathbf{q} .$$

This implies $D\mathbf{h} = \mathbf{o}$. Further, let

$$\begin{aligned} \bar{\mathbf{r}} &= C[\mathbf{x} + \mathbf{h}] - \mathbf{p} \\ &= \mathbf{r} + C\mathbf{h} . \end{aligned}$$

Then it follows that

$$\bar{\mathbf{r}}^t\bar{\mathbf{r}} = \mathbf{r}^t\mathbf{r} + 2\mathbf{r}^tC\mathbf{h} + \mathbf{h}^tC^tC\mathbf{h} .$$

The last term is non-negative, and the second term is zero since, under the assumption (2),

$$\mathbf{h}^tC^t\mathbf{r} = \mathbf{h}^tC^t[C\mathbf{x} - \mathbf{p}] = -\mathbf{h}^tD^t\mathbf{y} = 0 .$$

Hence, $\mathbf{r}^t\mathbf{r}$ is minimal. $\diamond$

Remark 11: If there are no constraints, i.e., $D = O$ and $\mathbf{q} = \mathbf{o}$, then the linear system (2) reduces to the so-called **Gaussian normal equations**

$$C^tC\mathbf{x} = C^t\mathbf{p} .$$

Remark 12: If C a is the identity matrix and if $D\mathbf{x} = \mathbf{q}$ is an underdetermined system, then (2) consists of the **correlate equations**

$$\mathbf{x} = \mathbf{p} - D^t\mathbf{y}$$

and the **normal equations**

$$D\mathbf{p} - DD^t\mathbf{y} = \mathbf{q} ,$$

which one obtains by substituting the correlate equations into the constraints $D\mathbf{x} = \mathbf{q}$.

Remark 13: Let W be an $m \times m$ diagonal weight matrix. Then, as a

consequence of the theorem above, the weighted residual

$$W\mathbf{r} = WC\mathbf{x} - W\mathbf{p}$$

becomes minimal under the condition $D\mathbf{x} = \mathbf{q}$ for the solution $\mathbf{x}, \mathbf{y}$ of the weighted equation

$$\left[\begin{array}{c|c} C^tW^2C & D^t \\ \hline D & O \end{array}\right] \left[\begin{array}{c} \mathbf{x} \\ \hline \mathbf{y} \end{array}\right] = \left[\begin{array}{c} C^tW^2\mathbf{p} \\ \hline \mathbf{q} \end{array}\right] .$$

4.8 Improving the parameter

The quality of an approximation can often be improved by a different choice of the knots u_i. Let $\mathbf{p}(u)$ be a curve approximating some points $\mathbf{p}_i$ at certain parameter values u_i. Further, let v_i be the parameter values of the points on $\mathbf{p}(u)$ closest to the $\mathbf{p}_i$. Then, in general, the u_i and v_i are different. Hence, a least squares fit based on the v_i would lead to a curve that lies closer to the $\mathbf{p}_i$ since the new set of curves over which one minimizes also contains $\mathbf{p}(u)$.

A simple method to determine the v_i from the u_i is to linearize $\mathbf{p}(u)$ at u_i and compute the closest point to $\mathbf{p}_i$ on the tangent of $\mathbf{p}$ at u_i, as illustrated in Figure 4.6 left. This means solving the equation

$$[\mathbf{p}(u_i) + \Delta_i\mathbf{p}'(u_i) - \mathbf{p}_i]^t\mathbf{p}'(u_i) = 0$$

for Δ_i, which gives

$$\Delta_i = [\mathbf{p}_i - \mathbf{p}(u_i)]^t \frac{\mathbf{p}'(u_i)}{\|\mathbf{p}'(u_i)\|_2^2} .$$

Then, $u_i + \Delta_i$ is an approximation for v_i.

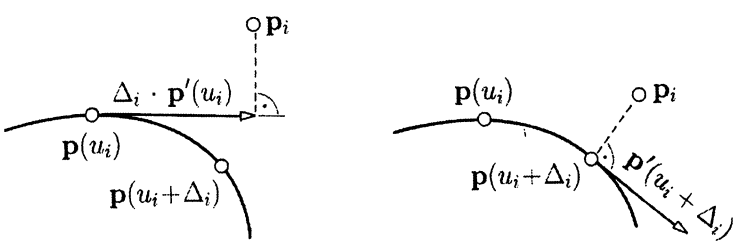

Figure 4.6: Improving a parameter.

Another method is to determine Δ_i such that the line from $\mathbf{p}_i$ to $\mathbf{p}(u_i + \Delta_i)$ is perpendicular to the tangent at $\mathbf{p}(u_i + \Delta_i)$, as illustrated in Figure 4.6 right. This is expressed by the condition

$$f(\Delta_i) = [\mathbf{p}(u_i + \Delta_i) - \mathbf{p}_i]^t \mathbf{p}'(u_i + \Delta_i) = 0 \ .$$

Using Newton's method, one obtains

$$\Delta_i \approx -\frac{f(0)}{f'(0)} = -\frac{[\mathbf{p}(u_i) - \mathbf{p}_i]^t \, \mathbf{p}'(u_i)}{\mathbf{p}'(u_i) \cdot \mathbf{p}'(u_i) + [\mathbf{p}(u_i) - \mathbf{p}_i]^t \, \mathbf{p}''(u_i)} \ .$$

One could iterate both processes to determine better approximations for the v_i, but usually one first computes a new approximating curve using the parameters $u_i + \Delta_i$ each time before doing so.

4.9 Problems

1 Use induction over n to show the **Hermite-Genocchi formula** for divided differences [de Boor '84, p. 21]

$$[u_0 \ldots u_n]\mathbf{q} = \int_{0 \le x_n \le \cdots \le x_1 \le 1} \mathbf{q}^{(n)}(u_0 + (u_1 - u_0)x_1 + \cdots + (u_n - u_{n-1})x_n) d\mathbf{x} \ .$$

2 Observe from the Hermite-Genocchi formula that

$$\lim_{u_0, \ldots, u_n \to u} [u_0 \ldots u_n]\mathbf{q} = \mathbf{q}^{(n)}(u)/n! \ .$$

3 Sketch the quintic Hermite polynomials $H_0^5, \ldots, H_5^5$ over some interval $[a, b]$ with their Bézier points.

4 For any three points $\mathbf{b}_0, \mathbf{b}_1, \mathbf{c}$ compute a point $\mathbf{b}_2$ on the line $\mathbf{b}_1\mathbf{c}$ such that the Bézier points $\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2$ define a quadratic curve $\mathbf{b}(t)$ with prescribed curvature $\kappa_0 = \| \dot{\mathbf{b}} \times \ddot{\mathbf{b}} \| \, / \, \| \dot{\mathbf{b}} \|^3$ at $\mathbf{b}_0$.

5 Compute and draw the interpolating polynomial which interpolates the **Gaussian error function**

$$\exp(-t^2/2) \quad \text{at} \quad t = -7, -5, -3, \ldots, 7 \ .$$

6 Compute and draw the piecewise cubic C^1 curve which interpolates $\exp(-t^2/2)$ and its derivatives at $t = -7, -1, +1, +7$.

7 Evaluate the interpolant of Problem 6 by de Casteljau's algorithm at $\pm 6, \pm 4, \pm 2, 0$.

8 Let $\mathbf{r} = C\mathbf{x} - \mathbf{p}$. Show that $\|\mathbf{r}\|_\infty = \max |r_i|$ is minimal for the solution $\mathbf{x}, \rho$ of the linear program

$$\begin{aligned} C\mathbf{x} - \mathbf{p} - \mathbf{e}\rho &\leq \mathbf{o} \\ -C\mathbf{x} + \mathbf{p} - \mathbf{e}\rho &\leq \mathbf{o} \\ \rho &= \min! \ , \end{aligned}$$

where $\mathbf{e} = [1 \ldots 1]^t$, see, for example, [Boehm & Prautzsch '93, 10.4].